

The Big Picture First

Think of the whole system as a relay race with 4 runners, each handing a baton to the next:

Raw crime reports → **[AI/ML extracts entities]** → **[AI/ML builds a graph]** → **[Database stores it]** → **[Backend serves it]** → **[Frontend shows it to an investigator]**

Every runner only needs to know two things: what shape the baton looks like when they receive it, and what shape it needs to be when they hand it off. That's the entire mental model — hold onto it, because everything below is just detail on top of this.

Piece 1: Entity Extraction (Tej — `extract.py`)

What goes in: Raw text. Example: *"Rahul met the owner of Sunil Traders in Sector 12, then called +91-9876543210."*

What happens inside: spaCy reads the sentence word by word and tags each word/phrase with a category it recognizes — this is pattern recognition it learned from millions of examples during its training (done by other people, long before you use it). It sees "Rahul" and recognizes the shape/context of a person's name. It sees "Sunil Traders" and recognizes an organization name pattern. For the phone number, since spaCy alone won't catch every format, a regex (a manual pattern rule like "10 digits starting with 6-9") catches it as a backup.

What comes out: A clean list:

```
[
  {type: "person", name: "Rahul"},
  {type: "organization", name: "Sunil Traders"},
  {type: "location", name: "Sector 12"},
  {type: "phone", name: "+91-9876543210"}
]
```

Why this matters for your pitch: This is the "unstructured to structured" transformation — turning messy human-written reports into clean, usable data. This is genuinely hard for a computer to do on its own without a trained model, which is why spaCy (not a simple find-and-replace) is doing this step.

Piece 2: Graph Construction (Shashwat — `graph.py`)

What goes in: The entity lists from *many* reports (not just one — hundreds of FIRs processed through Piece 1).

What happens inside: Every unique entity becomes a **node** (a dot). Then, the simplest rule: if two entities appeared in the *same report*, draw an **edge** (a line) between them. Do this across all reports, and patterns emerge naturally — if "Rahul" shows up connected to 5 different reports, each connecting him to different people, you start to see he's a hub, without anyone manually deciding that.

What comes out: A graph object, then exported as JSON:

```
{
  nodes: [{id: "n1", type: "person", name: "Rahul"}, ...],
  links: [{source: "n1", target: "n2", relationship: "co-occurred"},
  ...]
}
```

Why this matters for your pitch: This is the step that turns individual facts into a *network* — this is the actual "connecting the dots" that used to be done manually on a whiteboard.

Piece 3: Graph Analysis (Shashwat — still `graph.py`)

What goes in: The graph from Piece 2.

What happens inside: Two formulas run on top of the finished graph:

- **PageRank** counts, essentially, "how many important connections does this node have" — a node connected to many other well-connected nodes scores high. This flags **hubs**.
- **Betweenness centrality** measures "how often does the shortest path between two other nodes pass through this one" — a node that sits *between* two otherwise-separate clusters scores high. This flags **bridges/middlemen**.

What comes out: The same graph, but now every node has extra scores attached:

```
{id: "n1", name: "Rahul", pagerank: 0.34, betweenness: 0.61}.
```

Why this matters for your pitch: This is the actual "intelligence" — the system isn't just drawing a picture, it's telling the investigator *where to look first*. This is your

strongest technical talking point, because it's genuinely more than a database with a nice UI.

Piece 4: Storage (Piyush — Postgres)

What goes in: The final scored graph JSON from Piece 3.

What happens inside: Two tables hold this permanently:

- **nodes** table: one row per entity, with its type, name, and its PageRank/betweenness scores
- **edges** table: one row per relationship, pointing to two node IDs (foreign keys)

Why Postgres and not a specialized graph database: You already covered this — Postgres relational tables *shaped like* a graph, versus a true graph database (Neo4j). The tradeoff: Postgres is what the team knows, and the actual queries needed (direct connections, 1-2 hop lookups) don't require Neo4j's full engine.

Why this matters for your pitch: This is where the analysis becomes permanent and searchable — without this, every time someone wants to look something up, you'd have to re-run the entire AI pipeline from scratch.

Piece 5: Backend API (You — Express)

What goes in: Requests from the frontend ("give me the graph," "here's a new report to process").

What happens inside: Three jobs:

1. **Auth** — checks the investigator is logged in with a valid token before answering anything
2. **Ingest** — when a new report comes in, it forwards the raw text to Piece 1/2/3 (the Python AI service, called over HTTP), then takes the result and saves it via Piece 4 (the database)
3. **Serve** — when frontend asks "show me the graph" or "show me connections near person X," backend queries the database and sends back clean JSON

Why this matters for your pitch: This is the layer that makes the whole system *usable as a product* rather than a collection of scripts someone has to run manually. It's also your security layer — the only door in and out of the sensitive data.

Piece 6: Frontend (Shivam — React)

What goes in: The graph JSON from the backend.

What happens inside: A force-directed graph library takes the nodes/links and physically simulates them — connected nodes pull toward each other, unconnected ones push apart — so the layout naturally clusters related people together on screen. Nodes are colored by type (person/org/location) and sized by their PageRank score, so the "important" people are visually bigger without anyone needing to read numbers. Clicking a node opens a side panel showing that person's connections and — critically — *which reports* led to each connection (the evidence trail).

Why this matters for your pitch: This is what the judges actually see. A live, clickable, visually clustering graph is far more compelling in a 3-minute demo than a table of numbers, and the "why is this connection here" panel is what makes it trustworthy rather than a black box.

How to Explain the Whole Chain in 30 Seconds (memorize this)

"A report comes in as plain text. Our AI reads it and pulls out names, places, and numbers. It connects anything that appears together into a web. Two formulas then score every person in that web — one finds the most-connected person, one finds the middleman linking two groups. We store that scored web in a database, and our app shows it to the investigator as a live, clickable graph — click anyone, and you see exactly which reports proved that connection, so nothing is a black-box guess."

Why This Differentiates You From "Anyone Could Get This From AI"

The individual pieces are learnable/generatable, as you rightly pointed out. What differentiates you is: (1) you can explain *why* each piece exists and *why* that tool was chosen over the obvious alternative, (2) the evidence-trail panel — most teams will stop at "here's a graph," not "here's a graph you can actually trust and verify," and (3) a working, tested handoff between all six pieces — most teams generate the pieces and never properly test the seams, so theirs breaks live and yours doesn't.